BDD en la Práctica: Lenguaje, Colaboración y Automatización

sofka_

www.sofka.com.co

Índice

01 Introducción a BDD

02 Colaboración en BDD

03 Lenguaje y Estructura

04 BDD en la Práctica

05 Automatización con Cucumber

06 Flujo Completo de BDD en un Proyecto

sofka_

¿Que es BDD?

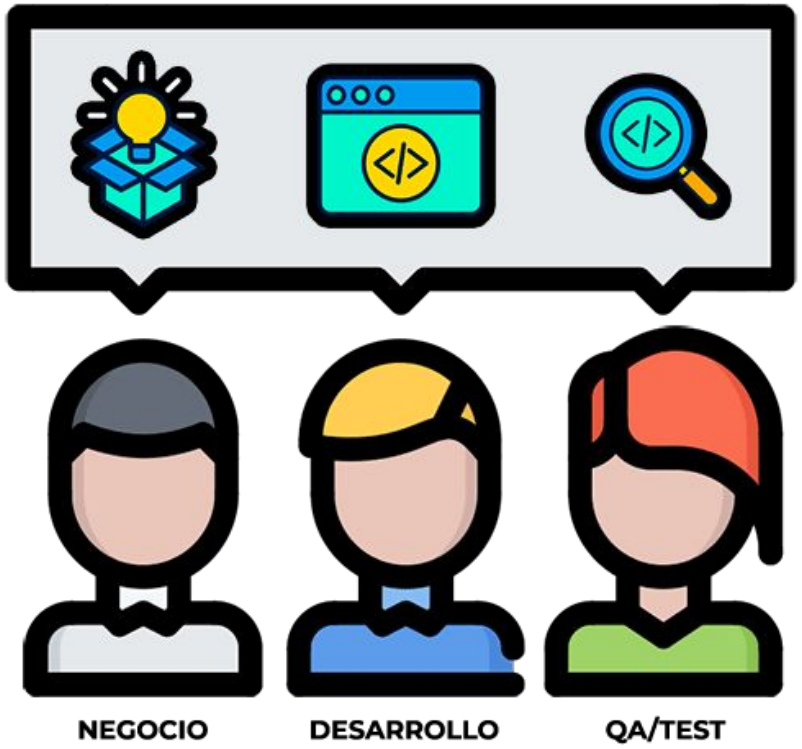
Behavior-Driven-Development: estrategia de desarrollo colaborativa que enfatiza el comportamiento de una aplicación para satisfacer las necesidades del negocio (Daniel North)

Propósito: Busca cerrar la brecha de comunicación entre los perfiles de negocio y los técnicos

Problema: No construir el software correcto

¿Que resuelve BDD?

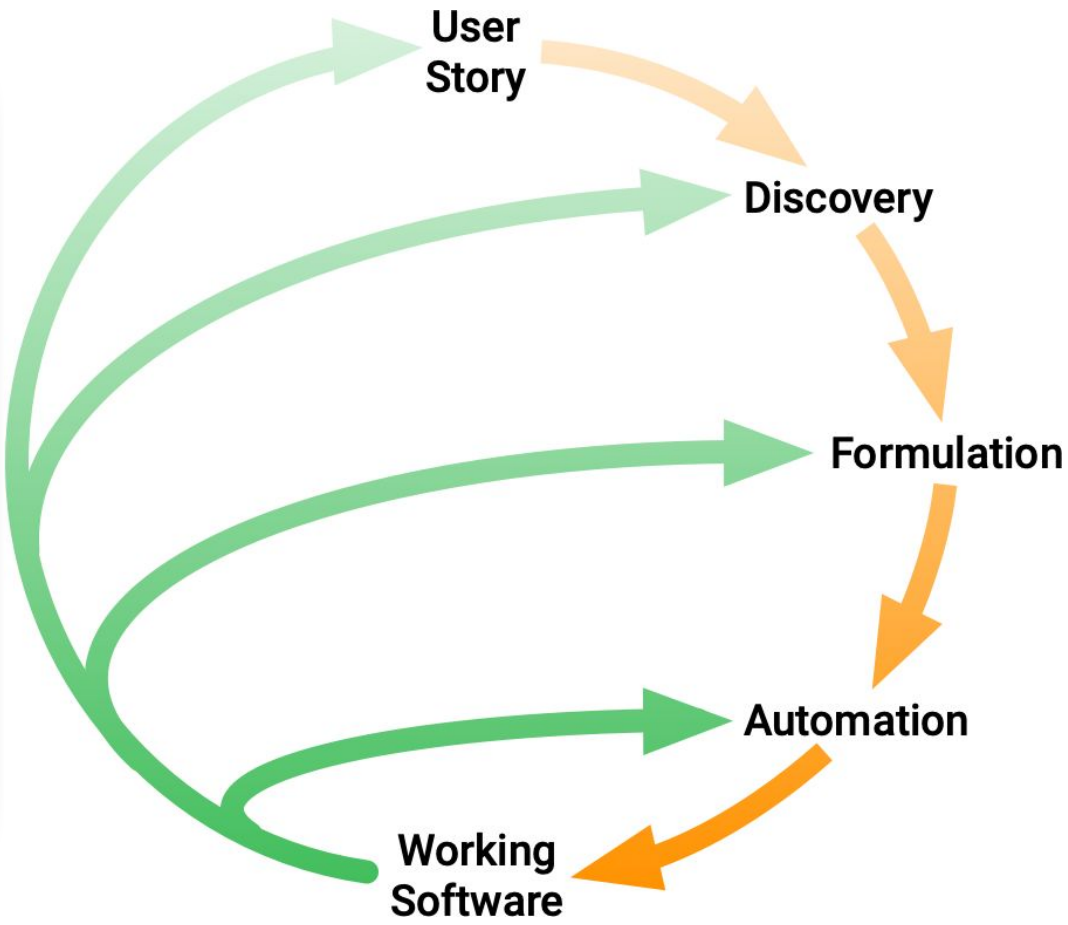
- Brecha de comunicación
- Falta de confianza
- Documentación obsoleta
- Malentendidos tempranos
- Construir el producto equivocado
- Baja testabilidad



Negocio, testers y desarrolladores crean los escenarios juntos de manera colaborativa a través de conversaciones.

¿Ciclo de BDD?

- **Discovery:** El equipo y stakeholders identifican **comportamientos** y resultados esperados
- **Formulation:** Identificados ejemplos valiosos se traducen en escenarios estructurados (Gherkin)
- **Automation:** Pasos técnicos que hacen ejecutables los escenarios anteriores



Mitos de BDD

- *"La automatización es la parte más importante de BDD"*
- *"Basta con usar Gherkin para decir que el equipo hace BDD"*
- *"BDD es solo una técnica de prueba (testing)"*

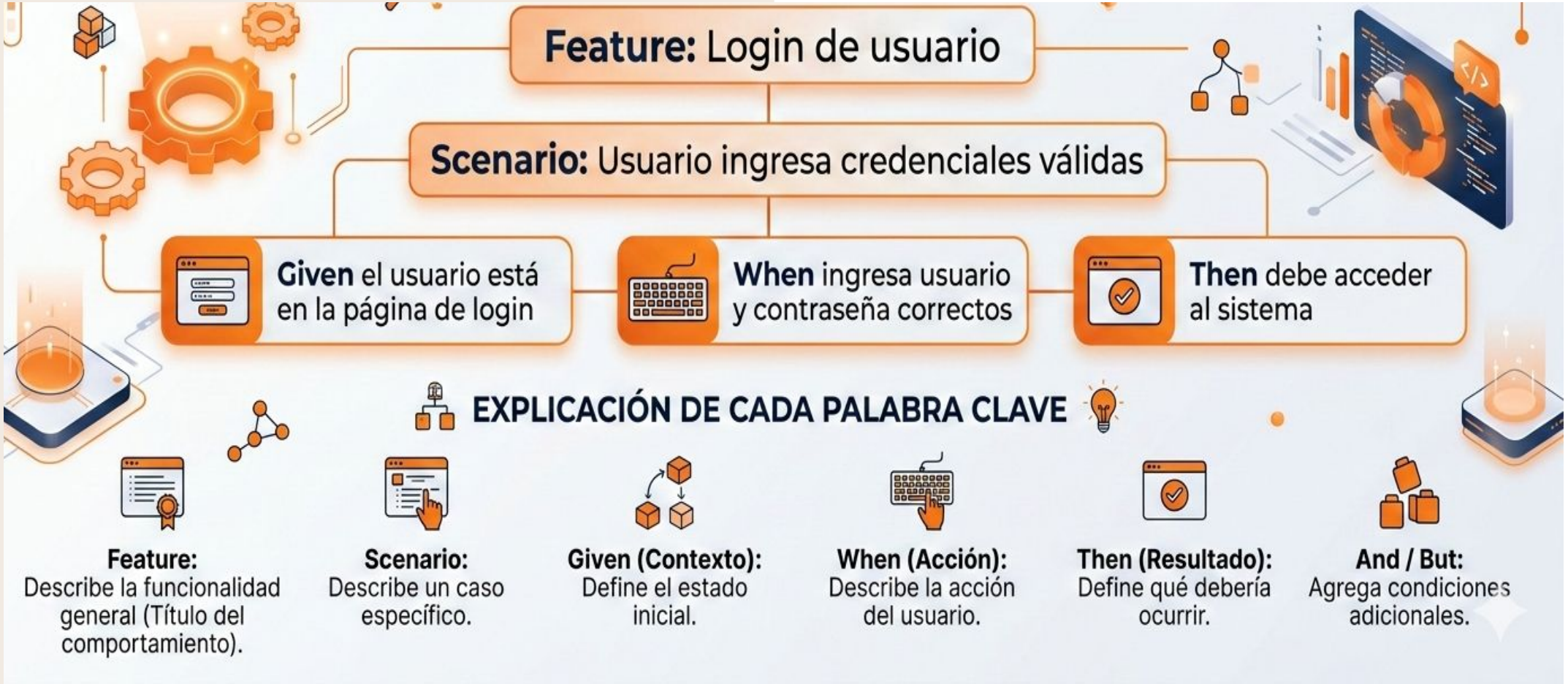
- *“Se pueden automatizar escenarios después de que el código fue implementado”*

03. Lenguaje y Estructura

03. Lenguaje y Estructura

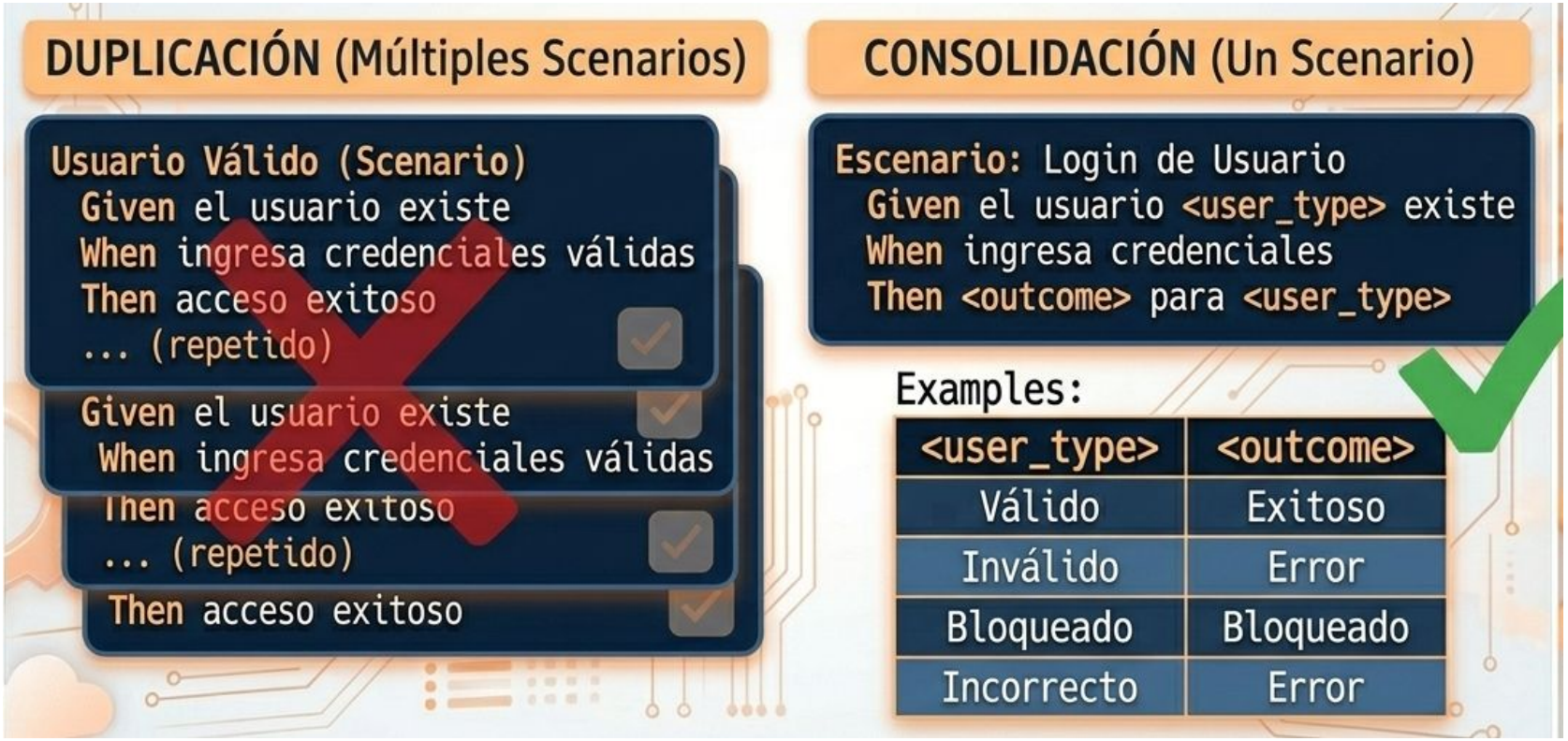
Gherkin se guarda en archivos ***.feature*** y es independiente de la implementación.

3. Cómo se estructura un archivo .feature



3. Scenario Outline y Examples

Cuando se quiere probar el mismo comportamiento con múltiples valores, se usa **Scenario Outline** con una tabla de Examples



3. Cómo organizar escenarios con GHERKIN

BACKGROUND (CONTEXTO COMÚN)

Definen pasos que son comunes a todos los escenarios en un Feature o Rule.

```
Feature: Gestión de Usuarios
  Background:
    Given el administrador está logueado
  Scenario 1: Crear Usuario
    ... (repetido)
  Scenario 2: Eliminar Usuario
    ... (repetido)
---
```

(Evita repetir contexto y mantiene archivos más limpios)

✗ DUPLICACIÓN DE PASOS

✓ REUTILIZACIÓN CON BACKGROUND

DOC STRINGS (TEXTO LARGO)

Para pasar bloques largos de texto como argumento.

```
Given a blog post with body:
  """
  This is long text
  that spans multiple lines
  and includes special
  characters, like quotes:
  "Hello, BDD!"
  """
```

(El texto se pasa íntegro al step definition)

DATA TABLES (LISTAS DE DATOS)

Para pasar listas o tablas de datos estructurados.

```
Given the following users exist:
| name    | email     | role  |
| Alice   | a@ex.com  | Admin |
| Bob     | b@test.com | User |
| Charlie | c@dev.com | Dev   |
```

(La tabla se pasa como un argumento estructurado)

Patrones vs Antipatrones

- Escritura **declarativa**
  Uso de **lenguaje de negocio**
- Escenarios **cortos y enfocados**
- Un escenario prueba **una sola regla de negocio**
- Títulos claros y orientados a valor
- Escenarios **independientes**
- Uso correcto de **Scenario Outline**
- Pasos consistentes
- personas no técnicas

- Escritura **imperativa**
- Uso de lenguaje técnico
- Escenarios largos
- Mezclar varias reglas de negocio
- Títulos genéricos
- Escenarios que dependen de otros escenarios
- Escribir Gherkin como si fuera un script de Selenium
- Enfocarse en la interfaz en lugar de la regla de negocio

Que es Cucumber ?

herramienta que permite la ejecución de especificaciones en texto plano para validar que el software hace lo que tiene que hacer

Features

```
Feature: Búsqueda en Google
  Como usuario web, quiero buscar en Google para poder responder mis dudas.

  Scenario: Búsqueda simple en Google
    Given un navegador web en la página de Google
    When se introduce la palabra de búsqueda "pingüino"
    Then se muestra el resultado de "pingüino"
```

StepDefintions

src/test/java/com/example/cucumber/steps/BusquedaSteps.java

```java
// Step definitions generated from BusquedaScenario.feature
package com.example.cucumber.steps;

import io.cucumber.java.es.Cuando;
import io.cucumber.java.es.Entonces;
import static org.junit.Assert.assertTrue;

public class BusquedaSteps {
    @Dado("un navegador web en la página de Google") public void unNavegadorWebEnLaPaginaDeGoogle() {
        // Lógica para abrir el navegador y navegar a google.com }   abrir el navegador y navegar a google.com
    }
    @Cuando("se introduce la palabra de búsqueda {string}") public void seIntroduceLaPalabraDeBusqueda(String palabra) {
        // Lógica para escribir '{string}' y presionar Enter, p.ej.: driver.findElement(By.name("q")).sendKeys(palabra + Keys.ENTER);
    }
    @Entonces("se muestra el resultado de {string}") public void seMuestraElResultadoDe(String resultadoEsperado) {
        // Lógica para verificar el título o los resultados, p.ej.: assertTrue(driver.getTitle().contains(resultadoEsperado)); }
    }
}
```

Posibles Escenarios

Undefined
Pending
Failing
Passing

¡Gracias!

Better
happens.
Every day.

El futuro no será definido por quién adopta IA primero, será definido por quién logra que su gente la adopte con confianza y responsabilidad. La ventaja competitiva ya no es sólo tecnológica, ahora es cultural.

sofka_

www.sofka.com.co